

Vector Embeddings Project

Building a Movie Recommendation System Like Netflix

In the previous lecture, we understood what vectors and embeddings are and why they are useful.

Now we will use those concepts to build a simple **movie recommendation system**.

The important point is:

| There is no ChatGPT deciding which movie to recommend.

We will build the recommendation engine using:

- Embedding Models
- Vectors
- Semantic Similarity
- Cosine Similarity
- Ranking

The goal is to understand how vector embeddings can be used in a practical application.

1. Quick Recap: What Is an Embedding?

Consider these three movie descriptions:

A: "An astronaut is stranded on Mars."

B: "A man must survive alone on another planet."

C: "A detective investigates a murder in London."

As humans, we immediately understand that:

A $\approx$ B

A is very different from C

But notice something interesting.

A contains words such as:

```
astronaut  
stranded  
Mars
```

B contains:

```
man  
survive  
alone  
planet
```

The sentences do not share many words.

Yet their **meaning is similar**.

This creates an important computer science problem:

How can a machine represent meaning in a form that mathematics can work with?

Computers are extremely good at working with numbers.

But they cannot directly perform mathematical operations on abstract ideas such as:

```
"These two sentences mean almost the same thing."
```

We therefore need a transformation:

LANGUAGE
↓
Numerical Representation
↓
MATHEMATICS

This is where **embeddings** become useful.

2. Representing Meaning as Coordinates

An embedding converts text into a numerical vector.

For example:

```
"A stranded astronaut tries to survive on Mars."
```

An embedding model may convert it into something like:

```
[  
  0.014,  
 -0.031,  
  0.087,  
  ...  
]
```

This array of numbers is called an **embedding vector**.

The vector represents information about the meaning of the input.

3. Does Every Dimension Have a Meaning?

A common misunderstanding is to imagine an embedding like this:

```
[space, survival, romance]
```

For teaching purposes, such an example is useful.

But real embeddings do not normally work like:

```
Dimension 17 = sadness  
Dimension 284 = space  
Dimension 611 = violence
```

The meaning is usually **distributed across the entire vector**.

Individual dimensions are generally not directly interpretable by humans.

The important idea is not what each individual number means.

The important property is:

Inputs with similar meanings tend to receive representations that are closer together in vector space.

4. What Is an Embedding Model?

A model whose job is to generate embeddings is called an **embedding model**.

Its basic job is:

```
Text  
↓  
Embedding Model  
↓  
Vector
```

Example:

```
"The astronaut opened the airlock."
```

may produce:

```
[-0.024, 0.182, ...]
```

The output is not another sentence.

The output is a numerical representation.

5. Is an Embedding Model an LLM?

A useful way to think about it is:

An embedding model is a language model specialized for representation, while generative LLMs are primarily designed to generate language.

Both systems may use neural networks and Transformer-based architectures.

Both may begin with something similar to:

```
"The Martian is amazing"
  ↓
Tokenizer
  ↓
Token IDs
  ↓
Neural Network
```

But their objectives are different.

6. Generative LLM vs Embedding Model

Generative LLM

Suppose we provide:

```
"The astronaut opened the..."
```

A generative model tries to estimate which token should come next.

Conceptually:

door → 0.31
hatch → 0.27
airlock → 0.19
window → 0.04

It chooses a token and continues the process repeatedly.

token → token → token → token → ...

Its final output is:

TEXT

Embedding Model

An embedding model solves a different problem.

Input:

"The astronaut opened the airlock."

The model is not trying to answer:

What token should come next?

Instead, its job is roughly:

Give me a useful numerical representation of this input.

So:

Input Text
↓

Embedding Model
↓
Fixed-Size Vector

Example:

```
[-0.024, 0.182, ...]
```

7. Can an Embedding Model Answer Questions?

Suppose we send:

```
What is the capital of India?
```

to a generative LLM.

It may respond:

```
New Delhi
```

An embedding model will not return that answer.

It will return something like:

```
[0.023, -0.193, 0.082, ...]
```

That is because the purpose of the model is different.

The embedding model represents information.

The generative model generates information.

8. Why Not Simply Use an LLM for Recommendations?

LLMs can absolutely recommend movies.

For example:

```
I loved Interstellar.  
Recommend three similar movies.
```

An LLM may respond with:

```
Arrival  
The Martian  
Gravity
```

So the question is not:

| Can an LLM recommend movies?

It can.

The important distinction is between:

```
KNOWLEDGE / GENERATION
```

and:

```
SEARCHING MY DATA
```

9. Scenario 1 — Generic Recommendation

Suppose the user asks:

Recommend movies similar to Interstellar.

A large language model may already know about Interstellar and thousands of other movies from its training data.

For this type of generic recommendation, using an LLM can make sense.

10. Scenario 2 — Recommendation From My Own Catalog

Imagine we have our own streaming platform:

CoderArmyFlix

Our database contains:

7,328 movies

Some of these movies might be:

- New
- Regional
- Independent
- Internal content
- Released after the model's training data

Now the requirement becomes:

Recommend the three most relevant movies, but only from our catalog.

The problem is now fundamentally different.

11. The Naive LLM Approach

One possible solution would be to send the complete catalog to the LLM:

```
Here are my 7,328 movies...

Movie 1...
Movie 2...
Movie 3...
...
Movie 7328...

The user likes Interstellar.

Recommend three movies.
```

Technically, some version of this could work.

But it creates several problems.

12. Problem 1 — Context Size

With ten movies, sending everything may not matter.

```
10 movies
```

But real applications can contain:

```
10,000 movies
100,000 products
5 million documents
```

Sending the entire dataset with every request becomes impractical.

The amount of data can become extremely large.

13. Problem 2 — Repeated Work

Imagine one million users search the platform.

With the naive approach:

```
User 1
+
Catalog
↓
LLM
```

Then again:

```
User 2
+
Catalog
↓
LLM
```

And again:

```
User 3
+
Catalog
↓
LLM
```

The movie catalog barely changes.

Yet the model keeps processing the same information repeatedly.

This is unnecessary work.

14. Think From First Principles

Ask:

| Which part of the system changes frequently?

Movie descriptions?

Usually not.

User queries?

Yes.

So we can divide our data into two categories.

Stable Data

Movie descriptions usually remain unchanged.

Therefore, we can process them once.

```

Movie Description
  ↓
Embedding Model
  ↓
Movie Vector
  ↓
Save It
  
```

Changing Data

The user's search query changes frequently.

Therefore, it is processed when the user searches.

```

User Query
  ↓
Embedding Model
  ↓
Query Vector
  
```

Then we compare:

```

Query Vector
  ↓
Mathematical Comparison
  ↓
Movie Vectors
  
```

We have converted repeated AI processing into reusable numerical representations.

15. Precomputation

This technique is called **precomputation**.

The principle is simple:

| If something expensive does not change often, calculate it once and reuse it.

For example:

```
Interstellar Description
      ↓
Embedding API
      ↓
Vector
```

If the description of Interstellar remains unchanged tomorrow, there is no reason to generate the same embedding again.

Instead, save:

```
Interstellar → Vector
```

and reuse it.

The same principle applies to every movie in our catalog.

16. Problem 3 — We Do Not Need Generation

Our actual problem is:

| Which movie is closest in meaning to this query?

We do not need the model to write paragraphs.

We do not need something like:

```
"The Martian would be an excellent choice because..."
```

We simply need values such as:

```
Movie A → 0.87 similarity  
Movie B → 0.81 similarity  
Movie C → 0.42 similarity
```

Then we sort the movies.

This is fundamentally a **ranking problem**.

Once meaning has been converted into vectors, ranking can be performed mathematically.

17. Think Like an Engineer

Do not start by asking:

```
Which AI model is smartest?
```

Instead ask:

```
What operation does my problem actually require?
```

For:

```
Summarize this article
```

Generation makes sense.

For:

Write an email

Generation makes sense.

For:

Which item in my dataset is closest to this query?

Embeddings + similarity comparison make a lot of sense.

18. Problem 4 — Cost and Latency

A generative model produces output sequentially:

```
token
↓
token
↓
token
↓
token
```

An embedding model only needs to return a numerical representation.

More importantly, once the movie embeddings are generated, comparing them does not require another LLM call.

Our application becomes:

```
User Query
↓
One Embedding Request
↓
Local Vector Comparison
↓
Results
```

The recommendation itself does not require a generative LLM.

19. Can We Combine Embeddings and LLMs?

Yes.

Consider a request:

```
"I want an emotional space movie  
where the protagonist is isolated."
```

First we perform semantic search:

```
User Query  
  ↓  
Embedding Search  
  ↓  
Retrieve Relevant Movies
```

For example:

```
The Martian  
Gravity  
Interstellar
```

After retrieval, we could optionally provide these movies to an LLM.

```
Retrieved Movies  
  ↓  
LLM  
  ↓  
Natural Language Response
```

The LLM could then generate:

```
"Based on your preference, The Martian would be a strong match..."
```

Each system now performs the job it is best suited for:

Embedding Model → SEARCH

LLM → GENERATION

This idea forms the basic foundation behind **Retrieval-Augmented Generation (RAG)**, which we will study separately.

20. Our Project

We will build a simple recommendation engine for a movie streaming platform.

Each movie contains two important properties:

```
title
description
```

Example:

```
Interstellar
```

```
"A team of astronauts travels through a wormhole
in search of a new home for humanity."
```

Another example:

```
The Martian
```

```
"An astronaut becomes stranded on Mars and must
use science and ingenuity to survive."
```

The system will expose two main features.

21. Feature 1 — Semantic Movie Search

Suppose the user does not know the movie name.

Instead, they describe the kind of movie they want:

```
"I want something about a person  
trying to stay alive far away from Earth."
```

A traditional keyword search may struggle.

Our movie description may contain:

```
stranded astronaut  
Mars  
survive
```

while the user's query contains:

```
person  
stay alive  
far away from Earth
```

The exact words are different.

But the meaning is similar.

This is exactly where semantic search becomes useful.

22. Semantic Search Flow

The complete flow is:

```
USER QUERY  
  ↓  
Embedding Model  
  ↓  
Query Vector  
  ↓  
Compare With Movie Vectors  
  ↓  
Calculate Similarity  
  ↓  
Sort by Similarity
```

↓
Return Top Movies

A possible result might be:

1. The Martian
2. Gravity
3. Interstellar

This is **semantic search**.

We are searching based on meaning rather than exact words.

23. Feature 2 — Similar Movie Recommendation

Now suppose the user has already watched:

Interstellar

and clicks:

Show Similar Movies

We do not need to create another text query.

Interstellar already has an embedding:

Interstellar → Vector

So we simply compare its vector with the vectors of all other movies.

Interstellar Vector
↓
Compare With Other Movie Vectors

↓
Calculate Similarity
↓
Sort
↓
Closest Movies

Possible result:

The Martian
Gravity
Arrival

This gives us a simple movie recommendation engine.

24. Search and Recommendation Are Almost the Same Problem

Notice the similarity.

Semantic Search

Query Vector
VS
Movie Vectors

Movie Recommendation

Movie Vector
VS
Movie Vectors

The source vector changes, but the underlying comparison mechanism remains almost identical.

This is an important engineering insight because the same similarity infrastructure can support multiple features.

25. What Type of Recommendation System Are We Building?

The system we are building is a:

Content-Based Recommendation System

The recommendation is based on the content of the movie.

In our case:

```
Movie Description
  ↓
Embedding
  ↓
Compare Meaning
  ↓
Similar Movies
```

For example:

```
Interstellar Description
  ↓
Embedding
  ↓
Similar Movie Descriptions
```

26. What We Are NOT Building

We are not implementing:

```
Users who watched Interstellar
```

also watched Movie X

That approach uses user behavior.

It is generally associated with techniques such as:

Collaborative Filtering

Collaborative approaches can use information such as:

User A watched Movie X
User A watched Movie Y

User B watched Movie X

Therefore:
Movie Y may be relevant for User B

Our project does not use this information.

27. Real Recommendation Systems Are More Complex

Production recommendation systems can combine many signals:

Movie Content
Watch History
Ratings
Clicks
Other Users' Behavior
Freshness
Popularity
Business Rules

Our demo intentionally isolates one concept:

| Recommendation using embedding similarity.

This makes it easier to understand the role of vector embeddings without mixing several recommendation techniques together.

28. Core Mental Model

Before thinking about Spring AI, APIs, controllers, or databases, remember four operations.

Step 1 — REPRESENT

Convert meaning into numbers.

```
Text
↓
Embedding Model
↓
Vector
```

Example:

```
"The Martian is about survival on Mars."
↓
Embedding Model
↓
[0.12, -0.43, 0.08, ...]
```

Step 2 — STORE

Save those vector representations.

Conceptually:

```
Interstellar → Vector
The Martian  → Vector
Gravity      → Vector
Arrival      → Vector
```

For our simple project, these can initially live inside:

```
List<Movie>
```

A production application may use specialized storage systems, but that complexity is not required for understanding the basic idea.

Step 3 — COMPARE

Take a starting vector.

It may come from a search query:

```
Query Vector
```

or from an existing movie:

```
Movie Vector
```

Then compare it against the other vectors.

The question becomes:

```
How close is this vector  
to every other vector?
```

Step 4 — RANK

Suppose our similarity calculation gives:

```
Movie A → 0.89  
Movie B → 0.84  
Movie C → 0.78  
Movie D → 0.21
```

Sort the results:

0.89
0.84
0.78
0.21

The movies with the highest similarity appear first.

This gives us both:

Semantic Search

and:

Content-Based Movie Recommendation

29. Overall Architecture

Our complete system can be understood as two phases.

Phase 1 — Prepare the Movie Catalog

For every movie:

```
Movie
↓
Take Description
↓
Embedding Model
↓
Movie Vector
↓
Store Vector
```

Example:

```
Interstellar
↓
```

```

Description
  ↓
Embedding
  ↓
Vector A

```

```

The Martian
  ↓
Description
  ↓
Embedding
  ↓
Vector B

```

We repeat this for the entire movie catalog.

Phase 2 — Search

When the user searches:

```
"I want a survival movie in space"
```

we generate an embedding for the query.

```

User Query
  ↓
Embedding Model
  ↓
Query Vector

```

Then:

```

Query Vector
  ↓
Compare Against Movie Vectors
  ↓
Similarity Scores
  ↓
Sort

```

↓
Top Results

30. Recommendation Flow

For recommendations, we already have the movie vector.

Example:

```
User Opens Interstellar
      ↓
Interstellar Vector
      ↓
Compare Against All Other Movies
      ↓
Similarity Scores
      ↓
Sort
      ↓
Return Most Similar Movies
```

Therefore:

```
SEARCH
Query → Embedding → Compare → Rank
```

while:

```
RECOMMENDATION
Existing Movie Vector → Compare → Rank
```

31. Why This Project Matters

This small project demonstrates an idea used far beyond movies.

The same pattern can be used for:

Products
Documents
News Articles
Courses
Jobs
Images
Support Tickets
Knowledge Bases
Code

Coder Army

Whenever the problem looks like:

Find the items in my dataset whose meaning is closest to this input.

the general architecture becomes:

```
REPRESENT
  ↓
STORE
  ↓
COMPARE
  ↓
RANK
```

That is the fundamental idea behind embedding-powered search and our movie recommendation system.

Final Mental Model

Do not think of the embedding model as a chatbot.

Think of it as a machine that converts meaning into coordinates.

```
Meaning
  ↓
Embedding Model
  ↓
Vector
```

Once information becomes vectors, we can use mathematics to answer questions such as:

```
Which movies are close to this query?  
Which movies are close to Interstellar?  
Which documents are closest to this question?  
Which products are semantically related?
```

The complete idea can be reduced to:

```
REPRESENT → STORE → COMPARE → RANK
```

For our movie application:

```
Movie Description  
  ↓  
Embedding  
  ↓  
Vector  
  ↓  
Similarity Comparison  
  ↓  
Ranking  
  ↓  
Recommendation
```

No generative LLM is required for the recommendation itself.

The recommendation comes from **vector similarity**.